



Performance Counters and DWT Enabled Control Flow Integrity

Ananda Biswas¹ · Zelong Li² · Akhilesh Tyagi¹

Received: 30 May 2021 / Accepted: 30 September 2021 / Published online: 5 November 2021

© The Author(s), under exclusive licence to Springer Nature Singapore Pte Ltd 2021

Abstract

Control flow integrity (CFI) attacks resulting from buffer overflow and return-oriented programming are common. The problem is particularly acute for legacy systems and IoT devices. Legacy industrial control systems are not supported with periodic security patches leaving them susceptible to attack vectors published over the system life span. IoT devices, on the other hand, are thin devices with limited resources. This rules out many of the traditional heavy-duty software countermeasures for the IoT world. In this research, we deploy hardware/software solutions to detect CFI attacks. Many IoT devices are based on Raspberry Pi boards. These boards include ARM Cortex A-53 (Pi 3) or Cortex A-73 (Pi 4) processors. These ARM Cortex processors contain hardware counters that can be programmed to count microarchitecture level events such as branch mispredictions. Since control flow anomalies resulting from buffer overflow or return oriented programming (ROP) modify the program execution, the microarchitecture level events counts diverge. For instance, number of instructions issued per cycle could differ due to different instruction level parallelism. Hence, a vector of most discriminating hardware counters can flag control flow anomalies. This paper focuses on embedded programs. Embedded program behavior is dominated by the main event loops and task/event handlers, which can be measured with performance counters. Lighter weight IoT devices, based on ARM Cortex M4 or M7, include DWT (Debug, Watch and Trace) module, but not performance counters. DWT contains a much more limited set of counters. We show that DWT counters can also detect CFI anomalies with somewhat lower accuracy. For legacy software, we insert the performance counters instrumentation hooks with direct binary editing of ELF files. The proposed anomaly detection mechanism is evaluated on ArduPilot Team (2016)—a popular autopilot software on a Raspberry Pi 3 with PMU and DWT. A self-navigation program is evaluated on an iCreate Roomba platform with an ARM Cortex M4 processor which contains a DWT but not performance counters. We are able to achieve 97–99%+ accuracy with 1–10 μ s time overhead per control flow anomaly check.

Keywords Control flow integrity · Performance monitoring unit · Debug and watch timer · Machine learning · Buffer overflow · Return oriented programming · Support vector machine

Introduction

Smart cities, communities, and environments are enabled by Internet of Things (IoTs) that sense and control our environment. IoT devices are networked computing devices that typically deploy embedded software. In addition to the traditional network mounted attacks such as buffer overflow, code injection, and return oriented programming (ROP), IoT devices are also vulnerable to physical access attacks. Eventually, most of these attacks either intend to gain root privilege through a shell spawned through buffer overflow or ROP attack or exfiltrate secret crypto keys to gain access to secret data, communication, or functionality. In this paper, we focus on the first type of attacks.

This article is part of the topical collection “Technologies and Components for Smart Cities” guest edited by Himanshu Thapliyal, Saraju P. Mohanty, Srinivas Katkoori and Kailash Chandra Ray.

✉ Ananda Biswas
biswas@iastate.edu

Zelong Li
zelongl@iastate.edu

Akhilesh Tyagi
tyagi@iastate.edu

¹ Electrical and Computer Engineering, Iowa State University, Ames, IA, USA

² Computer Science, Iowa State University, Ames, IA, USA

IoT devices often contain embedded processors with limited memory and computing power. Minimal operating systems provide fewer security features. Hence, stack protections such as stack layout randomization or address space layout randomization (ASLR) [14] may not be available. Virtual memory support may not exist. Without these protections, systems become more vulnerable to *buffer overflow*, *code injection*, and *return-oriented programming* attacks. This argues for a resource-efficient solution with the limited memory capacity, computing bandwidth, and energy availability constraints.

Control flow integrity violations caused by buffer overflow or ROP modify the observable microarchitecture state. For instance, the number of instructions executed can vary significantly due to stack stuffing in buffer overflow. Similarly, ROP causes a large number of return instructions to be executed without the corresponding call instructions in order to instantiate the gadgets. When a shell is spawned for the adversary to gain root privileges, it can take tens of thousands of extra clock cycles and instructions. The performance monitoring unit (PMU) and DWT count microarchitecture level events such as Level 1 data cache misses, number of execution cycles, and number of branch mispredictions. The goal is to find the subset of microarchitectural events supported by the performance monitoring unit (PMU) or DWT that are most discriminating between normal and anomalous executions. A data set of PMU and DWT counter vectors from normal control flow and anomalous control flow executions can be used to train a machine learning classifier. The machine learning model can then be used to classify control flow executions of certain granularity (100 instructions, 1000 instructions, or something else) into a normal or anomalous execution.

Typical software monitoring solution logs some set of events which are examined periodically for an anomaly. Software logging poses a high overhead in computation time as well as storage and energy needs. Moreover, it is easier to log system level events in software since those are easily visible. The microarchitecture level events, due to their finer granularity, seem to exhibit better correlation with anomalous executions than system level events. Microarchitecture level events are not visible for system level software capture. They are accessible only through a specific hardware engine like PMU. System and program level patching to prevent anomalous executions such as StackGuard [9], ASLR [14, 17], and CFI [1] are still feasible, to be deployed orthogonally to the PMU level monitoring. They do present a high overhead though, often not acceptable in the embedded IoT world.

Patching a program at source code level to insert machine learning training and testing data capture hooks is straightforward task. For legacy systems though, when only the binary might be available, this task is challenging. Many

binary editing tools exist such as ANGR [21, 24], IDAPro [13, 15], Symbol fishing/PLT-GOT infection/section hooking tools like RADARE2 [2], LIEF [25]. Most of these tools only provide partial solutions to the problem of PMU hook insertion. We develop a direct Executable and Linkable Format (ELF) file editing technique for this purpose.

Figure 1 shows an overview of the approach. A program execution footprint measured through PMU/DWT is collected. This footprint is analyzed by machine learning to decide if an execution was normal or anomalous.

Contributions: PMUs have been used for detecting anomalous execution behaviors in server and desktop processing environment. We present an evaluation of control flow integrity violation in embedded programs on light-weight embedded processors. The lower end embedded processors do not even include a PMU. We illustrate that somewhat similar anomaly detection behavior could be achieved with DWTs at a lower detection accuracy. DWTs are more universally available even in the lower end embedded processors. A hardware unit to capture the microarchitecture level event counts reduces the resource needs significantly compared to a software logging solution. The binary editing technique for ELF also provides an effective mechanism to reinforce legacy binaries.

Paper Organization: “[Related Work](#)” places our work in the context of hardware security using performance counters. “[Adversary Model](#)” provides an overview of buffer overflow and ROP attacks. An introduction to PMU and DWT capabilities is presented in “[PMU/DWT](#)”. “[Reading PMU Counters: Native Instructions vs. Perf_Event](#)” discusses the robustness and stability of the PMU data capture. Binary editing techniques for patching a targeted legacy binary are presented in “[Binary Editing](#)”. “[Experimental Results](#)” presents memory, computation, and prediction accuracy results.

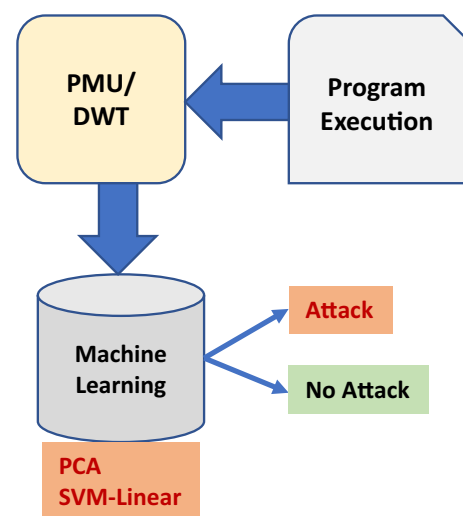


Fig. 1 Overview of the approach

“**Conclusions**” summarizes our approach motivation, challenge, solution, and novelty.

Related Work

Control flow integrity (CFI) was defined as a safety property in Abadi et al. [1]. Many more papers on CFI have been published in the mean time [8, 12, 16]. The main idea is to explicitly protect the targets of indirect branches—the control instructions that are hard to predict and have dynamically determined branch targets. Many types of protections are feasible. A statically inserted check against a set of profiled potential targets white list is a common technique. Hardware shadow stacks are also effective for return addresses. Our work also targets control flow integrity, but more at a control flow path level integrity through a path level cumulative signature.

Previous research [22, 26] has utilized PMU counters to gather application or kernel footprint to monitor for code reuse attacks or rootkits. These works fail to consider technique overhead and/or impact on system constraints. The work in [26] uses PMU counters to recover AES encryption keys. Samples of a dedicated time stamp counter (in x86 processor) and PMU counters configured to count L1/L2 data cache miss events are collected to demonstrate their attack technique. This replaces cache side channel with PMU readings.

Performance counters/performance monitoring units were originally intended to capture microarchitecture level events relevant to program performance. A set of hardware counters can be programmed to count one of many available events such as L1 D-Cache misses, branch mispredictions, retired instructions, and bus cycles. These units were originally intended to debug the microarchitecture. In early 2000’s, it was made available for performance tuning measurements for programmers. Some of the initial work on repurposing it for anomaly detection appeared in early 2010’s [19, 29]. Since then several papers have appeared on PMU deployment in program execution integrity detection [10, 11, 27, 28, 30].

Most of PMU driven integrity checks look at integrity of traditional OS modules. This typically involves collecting a PMU signature from beginning to the end of an OS service and checking it against an expected signature. Embedded programs have a common structure of an event loop which calls back into a small set of task handlers needing service in each iteration. Real time response requirements for some of these services complicate the issue further. These real time response constraints may be different over different sets of task handlers and different period event loops.

ROPecker [7] uses the last branch record (LBR) feature of x86 processors to detect code reuse attacks. LBR is a hardware structure which records the last 8 or 16 branch (source, target) address pairs. The data collection pre-processes

system libraries extracting information about potential gadgets. This information is stored in an off-line database. Code reuse attacks are detected during an on-line phase which uses both the database and LBR to look for potential gadget chains. Hardware performance counters are used in [22] to detect kernel rootkits. Counter value traces for various micro-architecture events are collected from executing code with and without rootkits. Machine learning techniques are used to classify traces into attack versus clean classes. The authors report an accuracy of 99% for detection of new rootkits with previously trained machine learning classifiers. This work is similar to our research in that machine learning is used to filter out micro-architecture events relevant to rootkit detection. However, the authors use off-line analysis and classification allowing unbounded storage and computational overhead.

We have developed a PMU/DWT counter based framework for attack detection in resource constrained embedded systems. Embedded system applications considered in this work are characterized by event loops with strict real time requirements. One such requirement is a strict bound on the maximum time required to execute the instructions in the loop. Further, these applications avoid using complex libraries to implement their functionality. Hence, most of the cited work is not applicable to embedded systems.

Adversary Model

Adversaries exploit program vulnerabilities. An exploit is the process of attacking a vulnerability. Vulnerabilities are any mechanism allowing adversaries to dictate instructions executed – program control flow.

Both buffer overflow and ROP adversaries manipulate program input. The malformed input is used to hijack program control flow.

Buffer Overflow: The program stack manages function calls. Each call pushes a stack frame or activation record. This stack frame keeps the return address of the caller at the bottom. All the local variables of the called function are also allocated in this stack frame. Consider the following program fragment:

```
function victim()
{
    char str1[20], str2[30];
    ...
    ...
    scanf("%s", str1);
    ...
}
```

In this function, char array `str1` is allocated on the stack frame for `victim()`. If an adversary provides an input much longer than 20 bytes allocated to `str1`, it overflows towards the return address `ra`, eventually overwriting it with the value consumed by `scanf`. Often the overwriting address also points to malicious code within the same string.

Modern operating systems protect against buffer overflow; stack canaries [5] and address space randomization [20] are such protections. Stack canaries are random numbers that are placed on the stack during a call. Returns ensure stack canaries are unchanged. Overwriting the return address requires overwriting the stack canary. Thus, the return address is protected. Address space randomization randomizes layout of the stack - hence location of `ra`.

OS will also disallow execution of data space - where program stack is embedded. Embedded processors often lack modern OS protections.

Return Oriented Programming (ROP) Once, data space is tagged as not executable by the OS, the executable payload cannot be supplied through buffer overflow. ROP instead exploits existing code, but in a different sequence. Gadgets are an instruction sequence ending in a return. The attacker identifies gadgets in a program code that provide the desired behavior. Many gadgets can be chained together through ROP. If `Gadget1` and `Gadget2` at addresses `ra1` and `ra2` provide the desired functionality for the adversary when chained as `Gadget1; Gadget2`; the buffer overflow part of ROP will push `ra2` followed by a push of `ra1` (`ra1` overwrites the original `ra` and `ra2` is right below it). The first return from the original victim function of the buffer overflow transfers control to address `ra1` or `gadget1`. Since `gadget1` ends in a return instruction, that return will transfer control to `ra2`, which executes `gadget2`—effectively chaining them. This technique can be used to chain n gadgets in general. Each return address on the stack causes one gadget to execute.

Goal of this work is to detect these buffer overflow and ROP attacks with high accuracy.

PMU/DWT

From the security monitoring perspective, PMU counters are akin to *big brother* in the micro-architecture world. These counters can be configured to *watch* executing application activities. They are a separate hardware unit enabling continuous overhead-free monitoring without posing an overhead on the application execution. Linux `perf` APIs provide user space access to performance counters via system calls to enable, disable, read, and write counter registers. PMU counters [3, 18] were originally intended to provide insight into performance bottlenecks. However, PMU counters have been extensively used in security research [26]. Surveying

relevant literature reveals PMU counter access to be a *double-edged sword*. It can be used to monitor an application state to detect malicious activity. On the other hand, exposing application state information, such as cache hits, creates a timing side channel that can be exploited in attacks. Timing analysis leaks information about micro-architectural events with asymmetric distributions (example—cache hit time for an evicted block, TLB miss for a page). In both cases, there is data collection (monitoring phase) followed by an analysis phase. Monitoring implementation specifics introduce trade-offs between resource overhead and technique accuracy.

ARM Cortex A53 PMU: The ARM Cortex A53 has a performance monitoring unit (PMU) that has one 64-bit cycles counter register and six programmable 32-bit event counters. This PMU supports 60 events such as L1 D-cache refill, L1 D-cache access, L1 I-TLB refill, Retired Load instructions, instructions retired, exceptions taken, instructions architecturally executed, mis-predicted branches, L2 D-cache access, Bus accesses, Conditional branches executed, cycles when the instruction issue queue is empty. These are just some of the representative events that can be counted. Figure 2 shows a schema for Cortex A53 PMU.

Configuration registers allow event selection—binding specific events to each of the six programmable performance counters. Counter registers increment when configured hardware events occur. The PMU can raise an interrupt when a specific count in the performance register is reached—which could be at each increment.

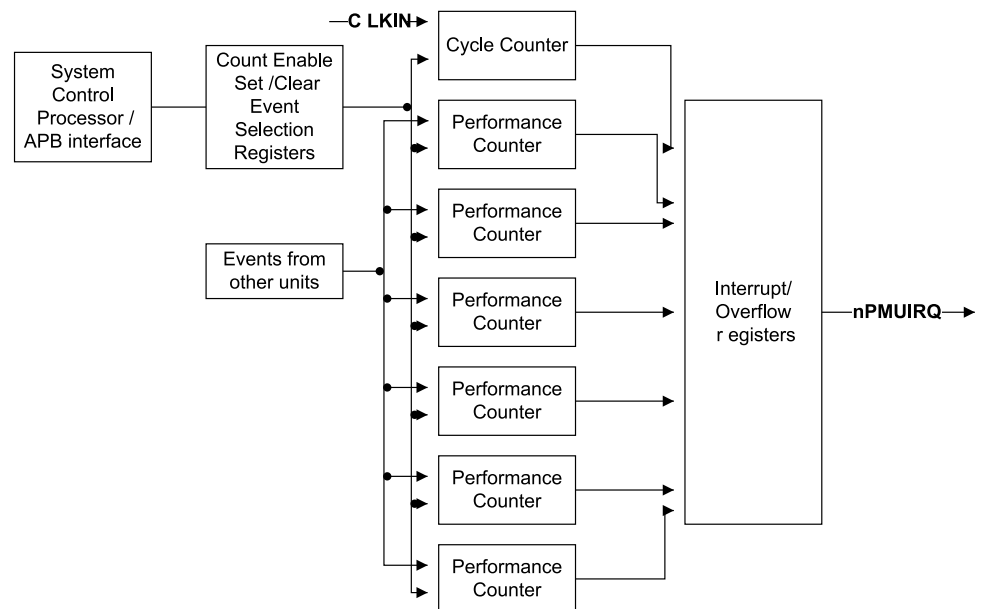
Linux perf: Linux `perf` is a performance monitoring package. Program monitoring tools through PMU are provided. `Perf` provides a set of virtualized events that can be synthesized from a combination of physical events in different processor microarchitectures. The raw event set supported by a microarchitecture can also be directly accessed. The `perf` package provides command line utilities and a system call interface.

`Perf` tools are integrated with the Linux kernel. This integration provides a system call interface to configure and read performance counters.

PMU counters are not virtualized on a per process basis by Linux. Hence PMU can continue collecting data across a process context switch and contaminate the data. `Perf` generates PMU event counts which are isolated per process through PMU register virtualization.

ARM Cortex M4 DWT: Data Watchdog and Trace (DWT) unit is primarily meant for debugging with limited support for performance tuning. The reason DWT is inferior to PMU is that PMU supports 60+ events instrumentation providing a rich feature set for machine learning based anomaly classification. A DWT will typically support 6-8 performance indicator events. The second more important difference is in the accuracy and delay of an event counter read. The event count capture in a PMU is *precise*—similar to a precise

Fig. 2 ARM cortex A53 PMU overview



exception—it can be precisely associated with an instruction. A `pmu-read()` instruction acts as a synchronization barrier. Just like in a precise exception, all the earlier instructions in the pipeline are allowed to finish, and all the following instructions are flushed without any committed state—including PMU state. Only then is a PMU event counter read. In complex pipelines such as an out-of-order superscalar microarchitecture, this flushing can take significant time. It is not unusual for a direct PMU register read to take of order of 100 clock cycles. In return, we get reproducibility in PMU read data for a `pmu-read()` at a specific location in the program.

A DWT `DWT-counter-read()` on the other hand takes only ten cycles or so. But it is not precise. It reads an event counter state that is affected by partially executed instructions in the pipeline as well. The ARM DWT manual [4] makes no guarantees of which instruction an event will be associated with. This results in somewhat noisy data which could lead to poorer classification accuracy for anomaly detection.

ARM Cortex M4 DWT has a dedicated 32-bit cycle counter. In addition there are other 8-bit performance event counters—specifically `DWT-INST-CYCLE-COUNT` which counts any extra cycles needed by an instruction beyond the microarchitecture design required cycles—typically stall cycles; `DWT-EXCEPTION-CYCLE-COUNT` counts all execution cycles from entry into an exception handler to exit from an exception handler; `DWT-SLEEP-CYCLE-COUNT` is the number of cycles in sleep; `DWT-LSU-COUNT` counts all the (stall) cycles needed by load/store instructions beyond necessary; `DWT-FOLD-COUNT` counts all the folded instructions that need zero cycles due to folding. Moreover,

there are other features of DWT that can also help broaden the machine learning feature space. There is a program counter (PC) sampling register which can be used to capture branch behavior. There are several comparison registers that can capture events based on a comparison against a specific data address, instruction address, or data value. These DWT signatures are relevant if certain instruction address, data address, or data value liveness property must be maintained within a code block.

The 8-bit performance counters count an event count N as $(N \bmod 256)$ which through aliasing can generate additional noise in the measurements. Our goal was to assess if despite all these shortcomings, can a DWT detect control flow anomalies within 10–20% of PMU detection accuracy.

Reading PMU Counters: Native Instructions vs. `Perf_Event`

The performance monitoring unit (PMU) available on modern architectures can be accessed by the user with native instructions or third party tools. On x86 architecture, the native `rdpmc` instruction is available to read the PMU counter state using the appropriate Model Specific Registers (MSR). ARM architecture provides a similar mechanism to read PMU counters using the `mcr/mrc` coprocessor instructions or `msr/mrs` for AArch64. In addition to these native architecture specific instructions, modern Linux operating systems (OS) provide a way to access these counters via the system call based `perf_event` interface. When deciding on a suitable method for capturing the PMU counter(s) state for our anomaly detection mechanism, we consider:

- How precisely can the PMU state be captured.
- What is the overhead involved in the capture mechanism.

We are interested in seeing which method offers higher precision in PMU data capture and the time overhead associated with it. There is a design space to be explored in this context. The timing overheads are limited by our target embedded system application. Higher precision in PMU state capture could mean better accuracy for our detection mechanism.

We begin our experimentation with native instructions. Our target system is an Arm Cortex A-53 (Raspberry Pi 3B) running Raspbian OS (Debian Jessie). We pick a program that performs matrix addition operation, $\text{Sum}[N \times N] = A[N \times N] + B[N \times N]$. We use three variations of the program, using the size of the matrices as 5×5 , 50×50 and 500×500 . The motivation behind using three versions of the program is to capture PMU counter states under different workloads to see how the event counters change with the workloads. We run each version of the program ten thousand times and calculate the mean and coefficient of variation for that data set. Read operations take less than ten cycles. This initial experimentation revealed that PMU data captured using native instruction has significant variance (Table 1). This is not desirable for our detection mechanism, as higher number of outliers in the data set could make the task of our machine learning model more challenging.

We speculate that since our testing program is run on an OS, it is likely that context switches occur during program execution which contaminates our PMU counter states. A context switch is an essential feature of multitasking operating systems that creates the illusion of multiple processes running simultaneously by swapping processes in and out of a core frequently. A context switch can be triggered in many different scenarios, such as switching between user and kernel modes or interrupts. During a context switch, the kernel suspends the currently active process on a core and starts executing a different process as determined by the scheduler. This involves saving the state (register, memory etc.) of the currently active process before swapping it out and restoring the saved state when it is brought back into the

core later. From literature survey and our own understanding of the PMU subsystem, we suspect that PMU registers state is not maintained during a context switch.

To verify this and find a way to capture cleaner data, we do a lightweight hack of the Linux kernel on the Raspbian OS. We modify the kernel to save and restore the PMU state on context switches. During a context switch, when the memory and register contexts are switched, we switch our PMU configuration and PMU counter states as well. We then boot into this hacked kernel to re-run our experiments. This time the data shows lower variance compared to the previous runs. Table 1 compares the mean and coefficient of variation values for data from the original kernel vs. our hacked kernel experiments.

Next, we test the `perf_event` subsystem. This is system-call based interface integrated into the Linux kernel (version 2.6.31 and newer). It is feature rich and can be configured in complex ways using the `perf_event_attr` structure. It is both similar and also very different from our hacked kernel approach. Both the hacked kernel and `perf_event` attempt to prevent contaminated counter state due to context switching. The `perf_event` virtualizes the event counters across context switches. Virtual event counters are defined independently of architecture supported physical events. The `perf_event` subsystem derives the virtual counter values from architectural physical event counters supported by a platform. The virtual counter state is also maintained per process. On the other hand, our hacked kernel approach still reads from the architectural PMU counter registers. They are different in the way counters are read. The `perf_event` uses `ioctl()`, `read()` system calls while our hacked kernel approach uses the native `mcr` instructions that is both time and resource inexpensive.

A third iteration of our experiment is run with `perf_event`. Read operations take about 300 cycles on average, which translates to less than a microsecond on a 1 Ghz processor. Table 2 compares the mean and coefficient of variation values for data collected from `perf_event` vs. our hacked kernel. We see that `perf_event` has a much lower coefficient of variation than our hacked kernel native

Table 1 Comparing retired instructions and cpu cycles measured on original kernel vs. hacked kernel

Matrix-size	Retired instruction count native instruction original kernel	Retired instruction count native instruction hacked kernel	CPU cycle count native instruction original kernel	CPU cycle count native instruction hacked kernel
5×5	$\mu = 889$ $\sigma/\mu = 0.768$	$\mu = 878.5$ $\sigma/\mu = 0.114$	$\mu = 1.2e^3$ $\sigma/\mu = 0.96$	$\mu = 1.2e^3$ $\sigma/\mu = 0.45$
50×50	$\mu = 7.3e^4$ $\sigma/\mu = 0.07$	$\mu = 7.3e^4$ $\sigma/\mu = 0.01$	$\mu = 9.8e^4$ $\sigma/\mu = 0.138$	$\mu = 9.7e^4$ $\sigma/\mu = 0.02$
500×500	$\mu = 7.6e^6$ $\sigma/\mu = 3.76$	$\mu = 7.2e^6$ $\sigma/\mu = 7.9e^{-4}$	$\mu = 1.1e^7$ $\sigma/\mu = 3.84$	$\mu = 1.1e^7$ $\sigma/\mu = 0.0053$

instruction experiments, of the order of 10's or 100's in certain cases. In terms of data capture precision, this seems to be the better candidate for our detection mechanism.

Determining the time constraints of our target program is critical. This will determine how much overhead we can afford. As a rough estimate, we decide that 5–10% of the program execution time is a reasonable overhead for our detection mechanism. Our target for anomaly detection is ArduPilot's [23] scheduler loop. This is discussed further in “[ArduPilot on ARM Cortex A53 PMU](#)”. We perform our PMU read operation followed by anomaly detection once per loop iteration. The loop itself attempts to complete a list of tasks within a given time quantum. By running the ArduPilot software multiple times, we determine that a loop iteration takes about 2 ms. Given this, `perf_event` read overhead of about 300 cycles is quite reasonable at less than 1% overhead.

Not all embedded systems can afford to run a full OS or have a high precision hardware PMU subsystem. Many use very lightweight hardware that runs only bare-metal programs. Some real time applications have even tighter time constraints. For such systems, the native instructions are a better fit for program state capture. The benefits of having a complex system like `perf_event` are more pronounced in a multithreaded environment. Bare-metal worlds have the benefit of cleaner performance counter states, since no other application can contaminate the data. In this work, we explore both the worlds.

Binary Editing

The proposed anomaly detection mechanism is also applicable to legacy systems where the system source code may no longer be available. We have developed methods that can be used to edit an existing ELF binary directly to insert our proposed mechanism into a legacy binary.

Executable and Linkable Format (ELF)

ELF is a common standard file format for executable files. It consists of an ELF header, program header table and section

header table. We can edit an executable file's ELF header and program header table to enable the executable file to include more executable instructions and their corresponding data so that we can insert our proposed detection mechanism into a legacy binary.

The ELF Header

For a 64-bit ELF binary, the size of the ELF header is 64 bytes. It is located at the beginning of an ELF file. The ELF header represents some important metadata of an executable file. For example, the entry point address attribute tells the operating system the instruction to begin the binary execution.

The Program Header Table

The program header table is an array of entries in which each entry is a structure describing a segment in the binary file or other information needed to create an executable process image. Usually a program header table contains several segments in which two are LOAD segments. The first LOAD segment is mostly filled with executable instructions and unchangeable constants. The second LOAD segment consists mostly of the global variables used in the executable binary. For each program header entry, it describes eight different attributes of a segment. It denotes the “type of a segment” such as LOAD segment. It has a offset and a “file size attribute” which tell the operating system which byte to start from in the binary file to load the data in this segment into the memory and how many bytes in the file should be loaded into the memory. The “virtual address attributes” denote the start of the virtual address of this segment when loaded into the memory. The “memory size attribute” sets the size of this segment in the memory. The “flags attribute” is used to control if a segment is readable, writeable or executable when loaded into the memory. A common program header table of an ELF file looks as below:

Table 2 Comparing retired instructions and CPU cycles measured using `perf_event` vs. hacked kernel (native instruction)

Matrix-size	Retired instruction count native instruction hacked kernel	Retired instruction count <code>perf_event</code>	CPU cycle count native instruction hacked kernel	CPU cycle count <code>perf_event</code>
5 × 5	$\mu = 878.5$ $\sigma/\mu = 0.114$	$\mu = 855.02$ $\sigma/\mu = 2.2e^{-4}$	$\mu = 1.21e^3$ $\sigma/\mu = 0.45$	$\mu = 1.25e^3$ $\sigma/\mu = 0.04$
50 × 50	$\mu = 7.3e^4$ $\sigma/\mu = 0.01$	$\mu = 7.06e^4$ $\sigma/\mu = 1.68e^{-5}$	$\mu = 9.7e^4$ $\sigma/\mu = 0.02$	$\mu = 9.68e^4$ $\sigma/\mu = 0.0035$
500 × 500	$\mu = 7.2e^6$ $\sigma/\mu = 7.19e^{-4}$	$\mu = 7.0e^6$ $\sigma/\mu = 1.2e^{-6}$	$\mu = 1.1e^7$ $\sigma/\mu = 0.0053$	$\mu = 1.05e^7$ $\sigma/\mu = 0.0082$

Type	Offset	VirtAddr	PhysAddr	
	FileSiz	MemSiz	Flags	Align
PHDR	0x0000000000000040	0x0000000000000040	0x0000000000000040	
	0x00000000000001f8	0x00000000000001f8	R	0x8
INTERP	0x0000000000000238	0x0000000000000238	0x0000000000000238	
	0x000000000000001c	0x000000000000001c	R	0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]				
LOAD	0x0000000000000000	0x0000000000000000	0x0000000000000000	
	0x0000000000257570	0x0000000000257570	R E	0x200000
LOAD	0x0000000000258068	0x0000000000458068	0x0000000000458068	
	0x000000000001b758	0x000000000002fa58	RW	0x200000
DYNAMIC	0x00000000002727e0	0x00000000004727e0	0x00000000004727e0	
	0x0000000000000230	0x0000000000000230	RW	0x8
NOTE	0x0000000000000254	0x0000000000000254	0x0000000000000254	
	0x0000000000000044	0x0000000000000044	R	0x4
GNU_EH_FRAME	0x0000000000215258	0x0000000000215258	0x0000000000215258	
	0x000000000000ae6c	0x000000000000ae6c	R	0x4
GNU_STACK	0x0000000000000000	0x0000000000000000	0x0000000000000000	
	0x0000000000000000	0x0000000000000000	RW	0x10
GNU_RELRO	0x0000000000258068	0x0000000000458068	0x0000000000458068	
	0x000000000001af98	0x000000000001af98	R	0x1

In order to patch a target legacy binary with the proposed mechanism, we create a separate patch file first. This is the prototype of the final patch file. This file includes the definitions of all PMU and machine learning initialization functions along with the PMU read, machine learning training and prediction functions. Note that for the PMU based anomaly detection, this patch is made available by us, and does not need to be rebuilt by the end-user. The proposed mechanism is first implemented in a C++ program. This program is compiled with the “-static-pie” flag enabled. This flag specifies to the g++ compiler to compile the patch’s source code into a statically linked *position independent executable* binary. The desired functions implemented in the patch file source code are called at least once in order for the compiler to include the binary code and its corresponding data in the object file.

This prototype patch file is concatenated with the target binary. This generates a patched target binary with all the instructions and data needed for the proposed mechanism. However, without changing the ELF and the program control

flow in the target binary, the content of the patch will never be loaded into the memory or invoked by the target binary.

We modified the program header table in the patched target binary to load the patch instructions and data into the target binary process image before the patched binary starts executing. This is accomplished by converting one of the legacy program header table entry into a “LOAD segment” that contains the patch content. We choose the NOTE program header table entry as it commonly exists in ELF files. A modification of this entry does not corrupt our target binary. By editing the bytes in the patched binary file directly in a binary file editor, we can convert a “NOTE program header” entry into a “LOAD program header” entry. The end effect is that all the content of the patch is loaded into this segment in the memory. We also changed the newly added LOAD segment to be readable, writeable and executable since the patch contains both instructions and data. An example of program header table after converting the NOTE segment entry into a LOAD segment entry is as follows:

Type	Offset	VirtAddr	PhysAddr
	FileSiz	MemSiz	Flags Align
PHDR	0x0000000000000040	0x0000000000000040	0x0000000000000040
	0x00000000000001f8	0x00000000000001f8	R 0x8
INTERP	0x0000000000000238	0x0000000000000238	0x0000000000000238
	0x000000000000001c	0x000000000000001c	R 0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]			
LOAD	0x0000000000000000	0x0000000000000000	0x0000000000000000
	0x0000000000257570	0x0000000000257570	R E 0x200000
LOAD	0x0000000000258068	0x0000000000458068	0x0000000000458068
	0x000000000001b758	0x000000000002fa58	RW 0x200000
DYNAMIC	0x00000000002727e0	0x00000000004727e0	0x00000000004727e0
	0x0000000000000230	0x0000000000000230	RW 0x8
LOAD	0x00000000002e5830	0x000000007f2e5830	0x000000007f2e5830
	0x00000000002e62b8	0x00000000002e7c70	RWE 0x200000
GNU_EH_FRAME	0x0000000000215258	0x0000000000215258	0x0000000000215258
	0x000000000000ae6c	0x000000000000ae6c	R 0x4
GNU_STACK	0x0000000000000000	0x0000000000000000	0x0000000000000000
	0x0000000000000000	0x0000000000000000	RW 0x10
GNU_RELRO	0x0000000000258068	0x0000000000458068	0x0000000000458068
	0x000000000001af98	0x000000000001af98	R 0x1

Other attributes of the header table entry that describe the segment that contains the patch contents are also modified accordingly to make the patched target binary execute correctly. The offset is set to point to the first byte that belongs to the patch file in the patched binary. The virtual address and memory size are set to be not overlapping with any existing segments in the target binary. Note, the memory size of the added segment is calculated as the patch file's 2nd segment virtual address plus the patch file's 2nd segment virtual address size. By default, an ELF executable binary contains two LOAD segments. These two segments will be loaded in two different virtual address ranges which are exactly two megabytes apart. Same situation also applies to our patch file. When the instructions in our patch file access data, they look for data using a relative address in the memory. However, when the prototype of the patch file is applied to the target binary, the whole patch will be contained in a single segment where instructions and data are adjacent to each other. In this case, if an instruction is trying to access the data, it is aware of the two megabyte gap between the instruction address range and the data address range. Such memory access will lead to an illegal memory access and cause segmentation faults. In order to solve this issue, we manually created the two megabyte gap by inserting two megabytes of zeros in the prototype of the patch file. This maintains the relative address offset between the instruction and the data when the patch is applied to a target binary.

Library Function Initialization

In the implementation of our proposed mechanism, many libc library functions are used and some of them require initialization before they can be called. This initialization procedure is done before the `main()` function starts executing. Our patch also needs to go through the initialization procedure although it is embedded in a legacy binary. The entry point address specified in the ELF header is the entry point of such initialization procedure. By modifying the target binary's entry point to point to the patch's initialization function, we can call the patch's initialization function as desired.

Not only do we need to call this initialization function, some modifications to the instructions are also needed. In this step of creating the complete patched binary, we utilized IDA Freeware to help us edit directly on the patched target binary. The normal control flow of an executable binary starts from `_start()`, followed by the `main()` function. Here the `_start()` initializes all the necessary variables for libc library functions in the memory. We modified the control flow so that the patched target binary starts from patch's `_start()` and then goes to the target binary's `_start()` and finally executes the target binary's `main()`. To achieve this, we created several code caves to alter the control flow in the patched binary.

Code Cave Creation

After modifying the entry point address in the ELF header of the patched binary, the patched binary will start its execution by calling the `_start()` function in the patch first. Our first code cave is intended to resolve the thread local storage (TLS) problem. During the `libc` function initialization for the patch, the `__libc_setup_tls()` function will iterate through each program header table entry of the patched binary to set up the TLS. However, the `__libc_setup_tls()` function that belongs to the patch in the patched target binary will mistakenly iterate through the program header table that belongs to the patched target binary. A code cave is created in order to help the `__libc_setup_tls()` function find the correct location of the program header table that belongs to the patch.

A code cave is a series of unused bytes in the address space of a process. These bytes are replaced to include custom instructions. We choose to put the code caves into the 2MB gap we created in the patch segment. On X86 platform, our first code cave is constructed as follows:

```
Patch_seg_phdr:
    lea    rax, byte_7F2E5870
    jmp    PMU_seg_phdr_back
```

This code cave, named `Patch_seg_phdr`, can help load the correct address of the program header table into the `rax` register and then jump back to the original `libc` initialization procedure. In order to jump to this code cave, some instructions are replaced by a `jmp` instruction in the original initialization procedure so that the program can jump to the code cave and get the correct register values. The last instruction in a code cave is always a `jmp` instruction to jump back to the original control flow and continue execution.

Our second code cave helps restore the program stack to the normal state. Since the initialization procedure that belongs to the patch will push some data onto the stack, this needs to be undone. After the initialization procedure finishes, the patched program needs to clean up the stack as if the initialization procedure never executed. A small code cave as shown below can accomplish this task on an X86 platform:

```
pmu_ml_glibc_stack_clean: follows
    add    rsp, 120h
    jmp    _start
```

After restoring the stack, the code cave jumps to the `_start()` function that belongs to the target binary to start the initialization process of the target binary.

During the `libc` initialization process on x86, the `__libc_setup_tls()` function will set up an area in the memory which is called the TLS. The addresses that fall into the TLS are intended to be accessed frequently. The `fs` register value is set to point to the TLS for faster memory access. In Linux, `syscall 158` is used to set the value in `fs` register. Many `libc` functions, such as `malloc()` and `free()` rely on the TLS to function correctly. Thus, `malloc()` function is not even available at the time the TLS is being set up, and instead `sbrk` and `brk` Linux syscalls are called directly to allocate memory for the TLS. The occupancy of the memory by the TLS that belongs to the patch is not known to the target binary. Thus, when the target binary calls `malloc()` function later during its execution, the `malloc()` function may allocate the memory space that is occupied by the patch's TLS. This may result in the target binary instructions overwriting the patch's address space eventually corrupting the patched binary. A code cave is created to call the `malloc()` function in the patched binary immediately after the TLS for the patch has been set up to solve this issue. With the correct size argument passed to the `malloc()` function, the existence of TLS that belongs to the patch is also known to the patched binary. Later when the patched target binary requests to allocate more memory through `malloc()` or other similar functions, the TLS memory space that belongs to the patch will be considered as occupied and thus will not be reallocated to cause memory conflict. During our experiments, we found that allocating 0x15000 bytes for the TLS through `malloc()` solves the memory conflict issue successfully.

Another problem on the x86 platform regarding the `fs` register is that the target binary also needs to have its own TLS in the memory. It is located in a different address space than the patch's TLS. This causes the problem that both the target binary and the patch want to set up the `fs` register to point to

their own TLS. This problem is solved by using the `gs` register which is also a segment register that is usually not used. We decided to let the patch use the `gs` register to point to its TLS leaving the `fs` register to the target binary. This is because the patch is usually smaller than the target binary which results in a smaller number of instructions that need to be modified in the patch to reference the `gs` register. Even though a smaller number of instructions are modified, it still is over six hundred instructions needing `gs` modification in the patch. This is the most time consuming part of the patch development through the ELF editing method. For the PMU anomaly patch, we have `gs` modification already expended that effort. The following shows an example of using the `gs` register in the patch instead of the `fs` register. Before modification:

```
mov     dword ptr fs:[rax], 16h
```

After modification:

```
mov     dword ptr gs:[rax], 16h
```

The only exception is when the instructions that belong to the patch reference the `fs:0x28`. Such instructions need not be modified to reference `gs:0x28` since it references a random number used as the stack canary for the stack protection.

In Linux, programs use a linked list to record the heap allocation. In our patched target program, the target binary and the patch each has its own heap linked list data structure. This causes memory allocation conflicts during the execution time since the same memory address may be allocated to both linked lists. We solve this problem by using `jmp` instruction as below:

```
malloc_patch:                                ;
            jmp     _malloc
```

In this example, `malloc_patch` is the `malloc()` function in the patch while `_malloc` is the `malloc()` function of the target binary. We replace the first instruction in the `malloc_patch` with a `jmp` to the target binary's corresponding functions. Since our example target binary is a dynamically linked binary, the example `jmp` is effectively jumping to the `.plt` entry of its `malloc()` function. By doing this, only one copy of `malloc()` is effectively used when the patched binary is executing. Thus only one linked list of memory allocation is used and the heap memory address conflicts will no longer occur. Similar modifications are also needed for `realloc()`, `calloc()` and `free()`.

The last step is to insert the PMU read and machine learning functions into our target binary to protect the target system. The code cave to start counting the PMU events is as follows:

```
task_handler_counter_start proc near
    push    rax
    push    rcx
    push    rdx
    push    rsi
    push    rdi
    push    r8
    push    r9
    push    r10
    push    r11
    call    counter_start
    pop     r11
    pop     r10
    pop     r9
    pop     r8
    pop     rdi
    pop     rsi
    pop     rdx
    pop     rcx
    pop     rax
    movzx   eax, byte ptr [rbx+7Ch]
    add     r13d, [rbx+78h]
    jmp     task_handler_counter_start_back
```

PMU State - Software Architecture

void AP_Scheduler::loop()

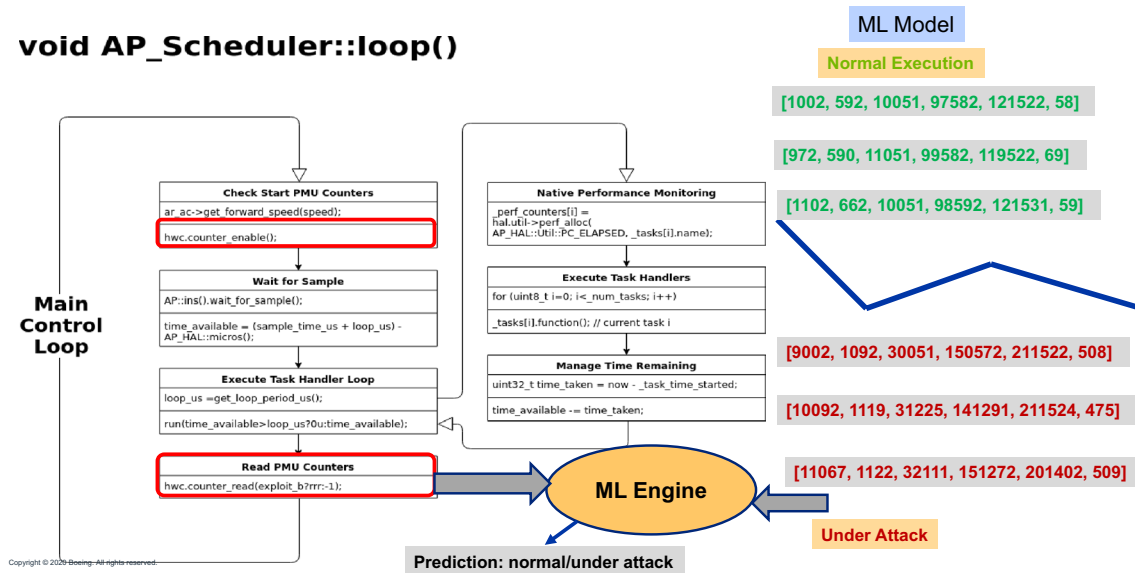


Fig. 3 ArduPlane software architecture

This code cave enables the PMU event counters. Later the PMU counter values are used by machine learning modules for normal or anomalous control flow prediction. Note that the `movzx` and `add` instructions at the end of code caves are from original target binary. Since these two instructions were replaced by a `jmp` instruction to jump to this code cave, they are placed in the code cave to preserve the patched binary behavior.

Automated ELF Editing and Patching

These steps patch a target binary by creating a brand new patch. However, to patch a target binary with an existing patch file, the process is much simpler. All that is needed is a target binary, a patch file, a patching script that can be implemented in many languages. A JSON script that contains the addresses of the bytes to be modified in the target binary is also used. We have created a script for automating the patching of our PMU/Machine learning patch into a target binary.

Experimental Results

We evaluated the proposed methodology with ArduPlane flight controller software on a Raspberry Pi and self-navigation software running on iCreate/RoboBee platform.

ArduPilot on ARM Cortex A53 PMU

We choose Raspberry Pi 3 together with Raspbian OS as our PMU testing platform. Raspberry Pi 3 is equipped with an ARM Cortex-A53 microarchitecture CPU that has PMUv3 integrated in it. A Navio 2 board is attached to the Raspberry Pi to receive all the sensor data that is required by ArduCopter program. ArduPilot can also be run in SITL (software in the loop) mode to derive the flight sensor data from a physical model. Some of our PMU data in this paper is from SITL runs of ArduPilot, and some of the data is from the ArduPilot runs with Navio board provided sensor inputs.

ArduPilot (ardupilot.org) is a popular open source flight controller software for fixed-wing autonomous or remotely piloted planes and quad-copters. It supports a variety of multiple three dimensional way-points, take-off and landing operations. It is a fairly complicated software enabling an IoT. Figure 3 shows the software architecture of ArduPlane. The main event loop is entered with a preallocated time quantum. Many tasks of various periods are polled to see if they need to be serviced. A worst case execution time (WCET) for the corresponding task handler is subtracted if a task handler is entered. The main event loop is exited if there are no more tasks or if there is no more time.

For our PMU testing platform, we use the ArduCopter program from the ArduPilot suite, a real world autopilot

system, to validate our framework. ArduCopter contains a main control loop that schedules a collection of task handlers to handle all the necessary tasks during its run time. We have inserted `pmu-read()` and `SVM-Linear-Predict()` functions at the end of the main event loop highlighted in red outline in Fig. 3. The `pmu-read()` function takes the difference between the current iteration and the previous iteration. This captures the PMU counter increment brought about by executing one main control loop iteration. During each iteration of the main control loop, different task handlers are picked by the scheduler depending on the priority of each task and the availability of the resources. Therefore, the PMU counters are stopped by calling `pmu-stop()` before task handlers are executed and restarted by calling `pmu-start()` at exit from a task handler. This collects PMU counters value associated with one iteration of the main control loop only.

We used LIBSVM [6] as our machine learning C library. The machine learning model is developed in advance off-line with 90,000 PMU reads for training data. We picked SVM-Linear since it gives us most efficient prediction with best accuracy. For the not-under-attack version of the ArduCopter, a data logging function is inserted in the `pmu-read()` for training purpose. For the under-attack version, besides the data logging function, a Buffer Overflow (BOF) attack which transfers the control flow to the adversary code is also inserted at various points in the ArduCopter program.

Training data are generated by running both the not-under-attack version and the under-attack version. This data is then provided to SVM to build a model.

A machine learning classifier is first trained with 20–30 combinations of 6 events out of 60+ supported by Cortex A53 PMU. The best 6 events combination in terms of prediction accuracy is adopted in the implementation. This is a physical limitation given that the PMU contains six physical event counter registers. This training occurs off-line. A trained classifier is then used to predict *attack* or *no attack* class.

Unlike the lightweight DWT, PMU has a significantly larger set of events that can be monitored. Perf classifies these events into six types, such as hardware, software and cache. Finding a good set of events to be monitored is very important especially when the size of the exploit is small. The process of finding a suitable event set is to look for the events that have a small $\sigma_{N_{loop}}/\mu_{N_{loop}}$ ratio. The standard deviation/mean ratio indicates how large the variance is as a fraction of the mean. A large variance makes it more likely that the two sets to be classified overlap creating confusion for the machine learning classifier. Hence, low S.D./Mean ratio events are considered to be less noisy and can help the machine learning model achieve a better accuracy. During the event selection process, we start from a set of six events and generate its corresponding training data. We then use this training data to train our SVM model. We then identify the event that has the least merit in helping our SVM model to classify the performance counter vectors. We replace this event with another available event and repeat this whole process again. In the end, we get a set of events that is most relevant to the classification of normal and anomalous control flow paths leading to a high accuracy. The PMU events we picked for our experiments in this paper are PERF-COUNT-HW-INSTRUCTIONS, PERF-COUNT-HW-CACHE-REFERENCES, PERF-COUNT-HW-CPU-CYCLES, PERF-COUNT-HW-BRANCH-INSTRUCTIONS, PERF-COUNT-HW-BRANCH-MISSES, and PERF-COUNT-HW-BUS-CYCLES.

Table 3 Tau-b Kendall correlation of event counters Aaross main event loop iterations

Counter #	tau (τ)
1	0.0901
2	0.0114
3	NA
4	NA
5	0.0338
6	0.0574

Table 4 SVM anomaly classification results with PCA

Linear SVM (C++ lib-SVM library)	Total sample size	Attack/non-attack sample size	Training/testing sample size	Training/testing accuracy
No PCA	317,359	158,576/158,783	158,679/158,680	99.7958%/99.7851%
6 PCA Components	317,359	158,576/158,783	105,786/211,573	99.8261%/99.8161%
5 PCA Components	317,359	158,576/158,783	105,786/211,573	99.8138%/99.8223%
4 PCA Components	317,359	158,576/158,783	105,786/211,573	99.7968%/99.8086%
3 PCA Components	317,359	158,576/158,783	105,786/211,573	99.7334%/99.7254%
2 PCA Components	317,359	158,576/158,783	105,786/211,573	93.934%/93.9175%
1 PCA Component	317,359	158,576/158,783	105,786/211,573	50.0321%/50.031%

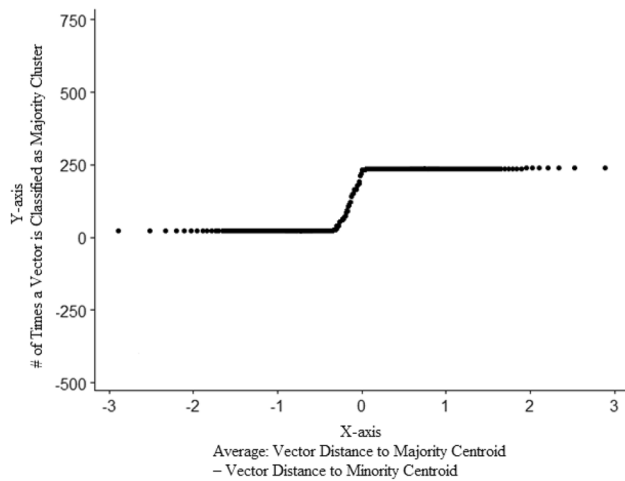


Fig. 4 Classification results for a set $S = \{v_0, v_1, \dots, v_{255}\}$ of normal vectors. x-axis shows the average of difference between majority and minority cluster difference $\sum_{\text{Clusters } C} (d(v, \text{majority_centroid}(C)) - d(v, \text{minority_centroid}(C))) / 256$. Frequency of majority cluster classification for raw vector v is shown on y-axis

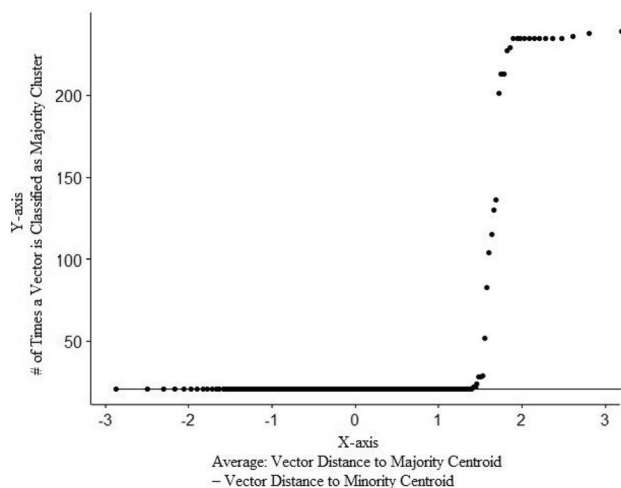


Fig. 5 Classification results for a set $S = \{v_0', v_1', \dots, v_{255}'\}$ of attack vectors. x-axis shows the average of difference between majority and minority cluster difference $\sum_{\text{Clusters } C} (d(v', \text{majority_centroid}(C)) - d(v', \text{minority_centroid}(C))) / 256$. Frequency of majority cluster classification for raw vector v' is shown on y-axis

Control Flow Path Classification Results

For an embedded main event loop as in Fig. 3, monitoring once per main event loop iteration is one natural design choice. We inserted the PMU counter capture instruction at the end of the main event loop. This PMU vector of seven counter values can be classified with machine learning based

on a model. We first address the question of the applicability of the same model to each loop iteration.

ArduPilot Loop Iteration Independence: The fact that we apply the same trained classification model for each iteration of the main event loop in Fig. 3 is based on a strong assumption that consecutive control loop iterations are uncorrelated. Note that ML classification is a statistical method. Kendall correlation analyzes consecutive iteration measurements for each counter. Kendall correlation coefficients are calculated based on the N th and the $N - 1$ th iteration pairs. For the counter #2 and #5 which have p values larger than 0.05 we fail to reject the null hypotheses and consider them as independent among each iteration. For the counter #3 and #6 which have p values smaller than 0.05, we reject the null hypotheses and consider them as not independent among each iteration. However, since the tau-b coefficient is minimal, we figured that the correlation is negligible. Tau-b, the corresponding correlation coefficients, are provided in Table 3. Tau-b value range is $[-1, 1]$ with values closer to the boundary suggesting strong negative or positive correlation respectively. Coefficients close to 0 suggest that the data are uncorrelated. Thus, consecutive ArduPilot loop iteration measurements of counter values are independent.

Supervised Models: We first assessed the effectiveness of machine learning on supervised models. In supervised learning, the data set is labeled into the classes, which in our case are normal and anomalous control flow paths. For training, when N PMU vectors are collected, the ratio of normal and anomalous PMU vectors can be controlled by controlling the frequency of anomaly injection. From the training data diversity perspective, a data set with roughly equal number of PMU vectors from both the sets is better. Hence, we collected roughly $N/2$ normal control flow path PMU vectors and $N/2$ anomalous control flow path PMU vectors.

Table 4 shows the classification results of SVM Linear on a large data set with $N=317,359$ for the main event loop of Fig. 3. The top row shows the training and testing accuracy when no principal component analysis (PCA) is used. The next six rows show the results when PCA with 6, 5, 4, 3, 2, 1 principal components is used. The classification accuracy is highest with six components.

Unsupervised Learning: Unsupervised learning does not train a model with both normal and anomalous behavior data sets. It trains a model only from normal execution traces. It's strength is in its ability to foresee anomalies that would not have been in a training data set, since anomalies are implicitly defined as any variation from the "normal" behavior. Unsupervised learning would often have lower accuracy than supervised learning.

We have used various approaches to unsupervised learning in order to support attack detection for the unknown attack types. First approach randomly selects 256 sets of 256 non-attack (normal) raw vectors. Each raw vector is a

concatenation of six 32-bit performance counters and one 64-bit cycle counter resulting in a 256-bit vector. For each set of 256 normal raw vectors, we train a k -means clustering unsupervised learning model with $k = 2$ resulting in a majority and a minority cluster. k -means clustering reduces a set of vectors into k clusters so that each vector belongs to the cluster whose mean (centroid) is closest to it. We get 256 such models/clusters with 2 centroids each, one for each training data set.

For testing, we use a similar collection of 256 sets of 256 vectors each. This approach works when the classification for anomaly is done after several execution runs. For each set of non-attack and attack raw vectors, the distance of each of the 256 vectors to the majority group and minority group centroids is computed. If the majority centroid is closer, the vector is classified as normal, if the minority group is closer it is classified as an anomaly/attack. Note that there are 256 (majority, minority) cluster models. This distance to majority cluster based classification against each of the 256 models results in a frequency of majority classification (between 0 and 256) for each raw vector under testing. Figures 4 and 5 show the distribution of majority cluster classification frequency for normal and attack vectors respectively. A given raw vector $\mathbf{v}$ gives one point in the graph in Fig. 4, where y-axis corresponds to the frequency of majority classification over 256 cluster models. For each raw vector, the Euclidean distance to every majority centroid d_{major} and every minority centroid d_{minor} is computed for each of the 256 cluster models. The x-axis represents the average value of $d_{\text{major}} - d_{\text{minor}}$ for a vector $\mathbf{v}$ over 256 models. To summarize, in Fig. 4, each point corresponds to (freq of majority classification, $\text{AVG}(d_{\text{major}} - d_{\text{minor}})$). Note that x-axis value of zero or above signifies that the raw vector is closer to the majority cluster on average. The majority cluster frequency classification is about 76 at 0 distance. Any majority frequency count of $76 + \epsilon$ can be construed to be a majority (normal execution) classification. Note that the min-frequency of majority cluster is the same for both normal and attack vectors (21) as well as max-frequency to the majority cluster is also the same at about 231. This is not much of a differentiation. The shape of distribution in Figs. 4 and 5 could be used for such classification, but we will rarely classify a group of PMU vectors at the same time.

Table 5 Exploit detection accuracy results for task handler #13

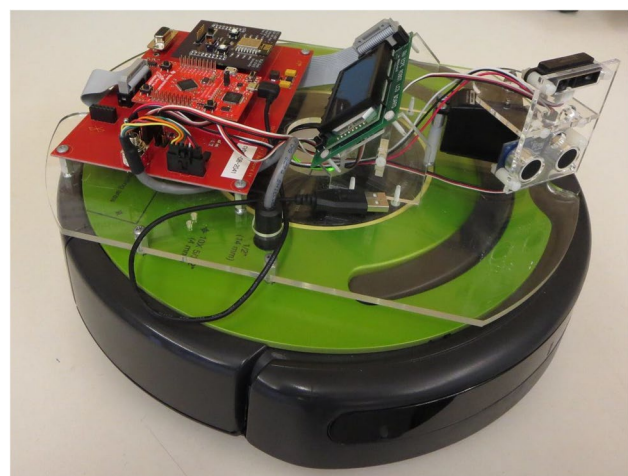
	6 PCA components	4 PCA components
Sample size	204,051	204,051
Attack sample size	102,029	102,209
Training accuracy	100%	99.9745%
Testing Accuracy	100%	99.9794%
Training time	0.221 s	0.409 s

The distribution based classification profile does not work for individual PMU vectors.

Second approach is to create a single model of majority/minority centroid, and classify a test vector based on its distance from the majority centroid. We created such a model with 1024 PMU vectors. We then classified 1024 normal test vectors and 1024 attack test vectors with this approach. 522 of 1024 normal test vectors were correctly classified as normal based on their distance to the majority centroid. This is not a very accurate classification for normal behavior. However, 1004 of 1024 attack test vectors were correctly classified as belonging to minority centroid. This seems to indicate that this approach might work well for abnormal behavior classification, but not so well for normal behavior classification. One model may be to use both this k -means based classification and our traditional linear SVM based classification in parallel. We will rely on linear SVM outcome for normal behavior, but on k -means outcome for attack classification. k -means distance computation is very minimal making such parallel classification approach practical.

Computation Time: The main event loop in ArduCopter typically starts with at least 0.1 s time quantum. Classification time is the time required to perform the prediction. With a C based SVM library, we are able to perform SVM Linear prediction for our model in 1–10 μ s. For Table 4, the classification time per PMU vector is 7.54 μ s, 6.48 μ s, 8.04 μ s, 9.8 μ s, 104.99 μ s, 224.93 μ s with 6, 5, 4, 3, 2, 1 principal components respectively. This shows the overhead of PMU based anomaly monitoring to be typically from 0.01 to 0.1%.

Buffer Overflow Exploit: For any two points A and B in the control flow graph, the change in PMU register state for a particular event is proportional to the number of times that



ARM Cortex M4-F processor
IR Sensor, Cliff sensor, ultra-sonic sensor

Fig. 6 iCreate/RoboBee platform

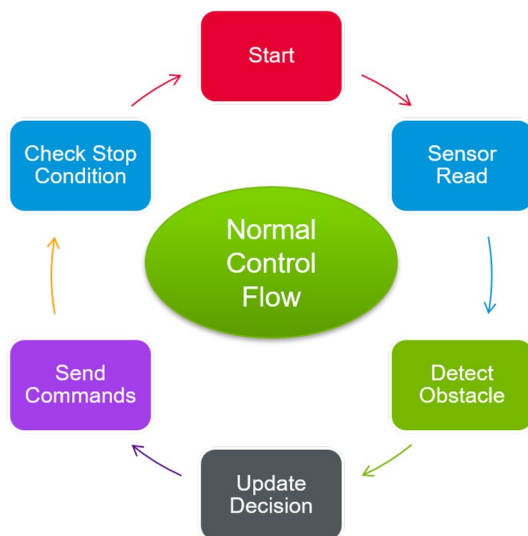


Fig. 7 Event loop structure of iCreate target program under normal control flow

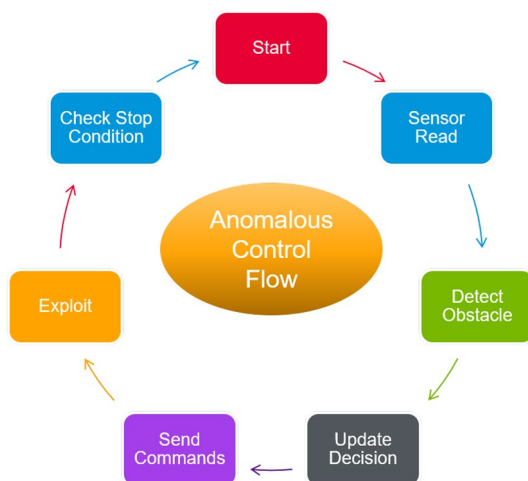


Fig. 8 Event loop structure of iCreate target program under anomalous control flow

event occurs between A and B. If we imagine A and B as the entry and exit points of our exploit gadget, then the state change of that PMU register is more profound if the event being captured occurs more frequently between A and B. Another important consideration in machine learning discovery of the exploit behavior is the granularity of exploit event count N_{exploit} in relation to the mean and standard deviation of the event count within single main event loop iteration $\mu_{N_{\text{loop}}}$, $\sigma_{N_{\text{loop}}}$. If $N_{\text{exploit}} \ll \sigma_{N_{\text{loop}}}$, it would be statistically impossible for machine learning to discover the exploit with high accuracy. We set up an experiment to determine

Table 6 Loop count vs. accuracy

Loop count (N)	Accuracy
0	50
1	60.9
2	65.8
3	63
4	70
5	85
6	86
7	89
8	94
10	94
12	95
14	96
16	98
18	98
20	99

how machine learning performs with varying sizes/granularity of exploit objects. Consider the following:

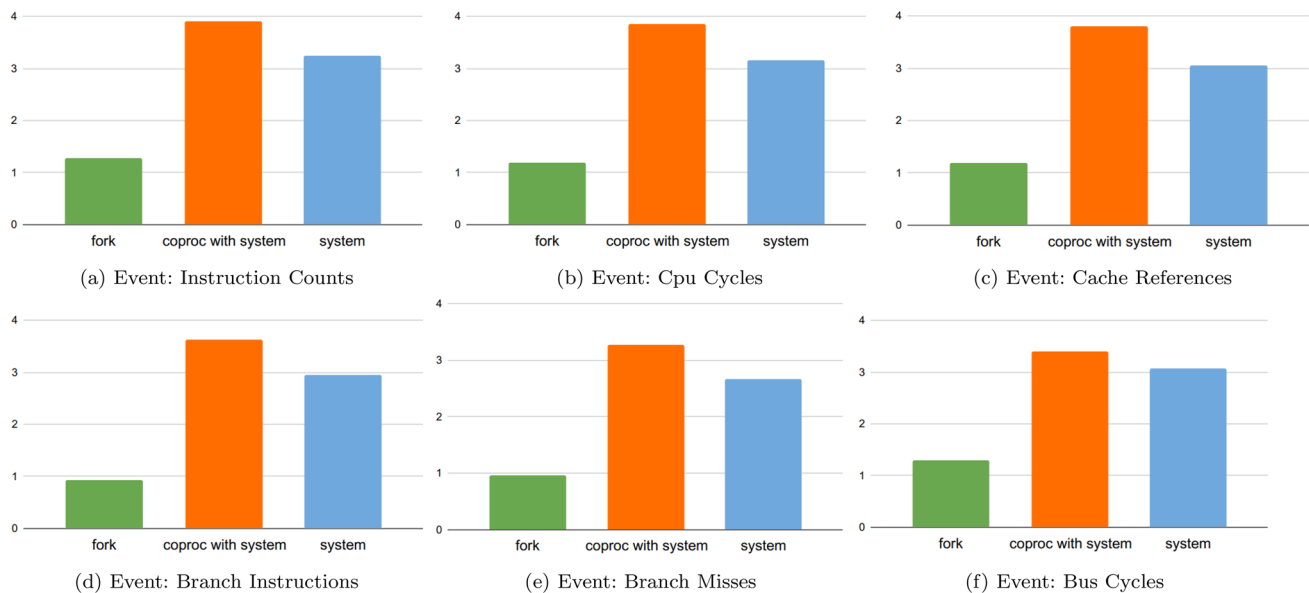
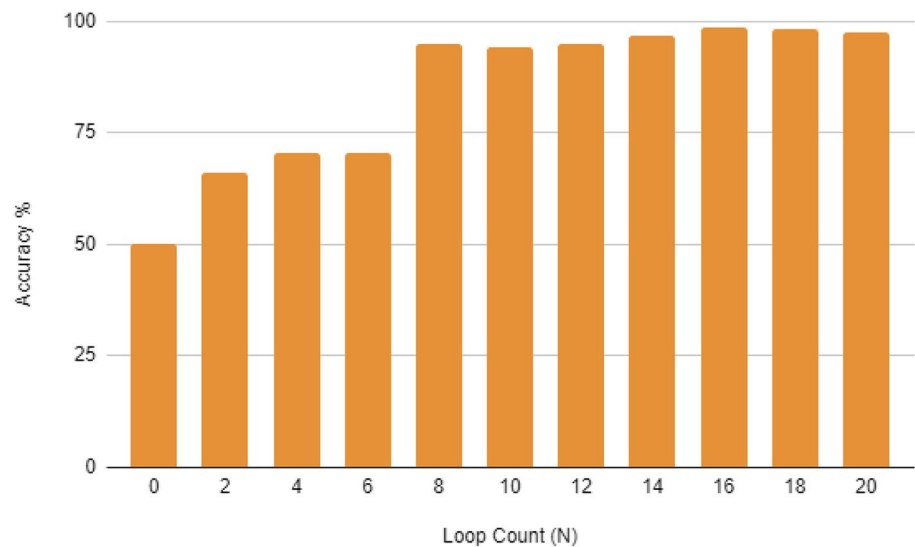
```

exploit()
{
  ...
  for(N times)
    printf()
  ...
}
  
```

Let **exploit()** be the exploit object. By varying the loop size N , we effectively increase the number of occurrences of events due to the `printf()` system call. We found that our machine learning classification accuracy increased with N as shown in Fig. 9. Table 6 tabulates the accuracy vs. loop count for this experiment.

Next, we implemented an exploit that is more likely to happen in a real world scenario: a buffer overflow that spawns a shell to allow the adversary to gain access to the victim system. The shell is spawned as a non-blocking child process. We found that the shell opening and exit immediately cause sufficient PMU state change for our machine learning classifier to classify with 99% accuracy.

Task Handler Measurement: We also did similar PMU measurements of various task handlers with `PMU-read()` at the entry and exit from a task handler. Actual task handler event count is the difference of the two readings. Similar ML training was performed. The trained model was used to make prediction about anomaly. Detection accuracy varies

Fig. 9 Classification accuracy with varying loop count**Fig. 10** Impact of different system calls on PMU state as a fraction of base case

from 99.97 to 100% for Task handler #13 as an example. The corresponding data is presented in Table 5.

Experimental Setup: DWT on iCreate Platform

We used an iCreate Roomba platform shown in Fig. 6 used in our embedded systems laboratory. It uses Texas Instruments TM4C123G Tiva Launchpad Evaluation Board as a controller board. This board contains ARM Cortex M4-F processor with DWT. The sensors used in this platform for self-navigation include IR, Ultrasonic, cliff sensor, battery sensor, and virtual walls. A self-navigation program that steers the platform towards a specific point in the room

avoiding cliffs created through removed floor tiles was evaluated for DWT control flow anomaly detection.

We picked an obstacle avoidance program as the target program for the iCreate platform running an Arm M4 processor. At the core of the program is a loop that iterates over various tasks infinitely, until a stop condition is reached. Figure 7 shows the structure of this event loop. The tasks include reading various sensor data, making navigation decisions based on the data read and sending commands to motors. Just like our PMU based experiments, DWT initialization and setup happens outside the event loop and does not corrupt the measurements.

A control flow anomaly was introduced to this program by adding extra functionality at the end of **Send Commands** stage. Unlike our PMU experiments where both the buffer overflow and instructions within the exploit gadget alter the PMU state, here DWT state changes due to exploit gadget alone. Note that the footprint of exploit gadget alone without the stack stuffing setps is smaller. Hence accuracy numbers for the exploit gadget detection form a lower bound on the detection accuracy for the full exploit.

Figure 8 shows the structure of the event loop under anomalous control flow. Based on 2000 loop iterations data with anomalous control flows occurring at 50% frequency, our machine learning model is able to classify anomalous behavior with 99% accuracy.

We further experimented with different exploit models to see how they impact the PMU state. In addition to shell exploits, adversaries may attempt to interact with the OS with other forms of system calls. A non-blocking `fork()` is one possible way an adversary may attempt to execute an action on the processor without hindering the execution of the original process. We also tested the `system()` command by spawning a coprocess using `system()`. We picked our base case as a buffer overflow exploit that does not do anything significant other than routine stack push/pop at the payload function boundary. The three cases we tested behave as follows:

- call `fork()`, child exits immediately
- call to `system()` with a bash script that spawns a coprocess
- call to `system()` with a null command (colon:)

We ran these experiments on a test program that uses the same exploit code as our ArduPilot based PMU experiments and on the same Raspberry Pi 3b platform. Figure 10 shows our results for this experiment. The graphs are plotted on a log base ten scale and represent the ratio of the test case to the base case for a particular PMU event. We observe similar patterns on all of the 6 PMU events. Fork() seems to be the most lightweight among the 3 but is still significantly higher than the base case, and hence much more easily detectable than the base case.

Conclusions

IoT devices frequently have different security design space than desktop or server systems. The embedded processors are computing, memory, and energy limited. Bare metal systems lack an OS, which rules out many modern security mechanisms. In this work, we explore the security design space for control flow anomaly detection with efficient and

real time hardware support for microarchitecture event logging with a PMU/DWT.

We explored control flow integrity of ArduPlane software on a Raspberry Pi 3 which contains an ARM Cortex A-53 processor with PMU support. We also evaluated a self-navigation program on an iCreate/Roomba platform with an ARM Cortex M4F processor with DWT support.

We are able to achieve 97-99%+ accuracy with $1 - 10\mu\text{s}$ time overhead per control flow anomaly check.

There are many other possible applications of PMU and DWT measurements in IoT security model. In CFI framework, labels used to tag an indirect target could be derived from actual PMU measurements. PMU/DWT values can even serve as a nonce or a random value, which is program context specific, which has many applications within the embedded world. We are exploring these directions.

Acknowledgements This material is based upon work supported by the Office of Naval Research under Contract No. N68335-17-C-0208. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Office of Naval Research.

Funding This study was funded by the Office of Naval Research (grant number N68335-17-C-0208).

Declarations

Conflict of Interest The authors declare that they have no conflict of interest.

References

1. Abadi M, Budiu M, Erlingsson U, Ligatti J. Control-flow integrity. In: Proceedings of the 12th ACM Conference on Computer and Communications Security, CCS 05, p. 340–353. Association for Computing Machinery, New York, NY, USA; 2005. <https://doi.org/10.1145/1102120.1102165>.
2. Alvarez S, Jin M. The official radare 2 book. <https://book.rada.re/> (2016). Accessed: 2021-05-22.
3. ARM. Arm cortex-a53 mpcore processor technical reference manual. <https://developer.arm.com/documentation/ddi0500/d/performance-monitor-unit/pmu-functional-description>. Accessed: 2020-08-15.
4. ARM. Armv7-m architecture reference manual, section c1.8 - dwt. <https://developer.arm.com/documentation/ddi0403/ed/>. Accessed: 2020-08-15.
5. Baratloo A, Singh N, Tsai TK, et al. Transparent run-time defense against stack-smashing attacks. In: USENIX Annual Technical Conference, General Track; 2000. p. 251–262.
6. Chang CC, Lin CJ. LIBSVM: A library for support vector machines. ACM Trans Intell Syst Technol. 2011;2:27:1–27:27. Software available at <http://www.csie.ntu.edu.tw/~cjlin/libsvm>. Accessed 8 Oct 2021.
7. Cheng Y, Zhou Z, Miao Y, Ding X, Deng HR. Ropecker: a generic and practical approach for defending against rop attacks. In: In Symposium on Network and Distributed System Security (NDSS); 2014.

8. Christoulakis N, Christou G, Athanasopoulos E, Ioannidis S. Hcfi: Hardware-enforced control-flow integrity. In: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy, CODASPY 16, pp. 38–49. Association for Computing Machinery, New York, NY, USA; 2016. <https://doi.org/10.1145/2857705.2857722>.
9. Cowan C, Pu C, Maier D, Hintony H, Walpole J, Bakke P, Beattie S, Grier A, Wagle P, Zhang Q. Stackguard: automatic adaptive detection and prevention of buffer-overflow attacks. In: Proceedings of the 7th Conference on USENIX Security Symposium - Volume 7, SSYM98, p. 5. USENIX Association, USA; 1998.
10. Das S, Werner J, Antonakakis M, Polychronakis M, Monrose F. Sok: The challenges, pitfalls, and perils of using hardware performance counters for security. In: 2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19–23, 2019; 2019. p. 20–38. IEEE. <https://doi.org/10.1109/SP.2019.00021>.
11. Demme J, Maycock M, Schmitz J, Tang A, Waksman A, Sethumadhavan S, Stolfo S. On the feasibility of online malware detection with performance counters. In: Proceedings of the 40th Annual International Symposium on Computer Architecture, ISCA 13, p. 559–570. Association for Computing Machinery, New York, NY, USA; 2013. <https://doi.org/10.1145/2485922.2485970>.
12. Diatchki I, Pike L, Erkök L. Practical considerations in control-flow integrity monitoring. In: 2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops; 2011. pp. 537–44. <https://doi.org/10.1109/ICSTW.2011.27>
13. Eagle C. The IDA. Pro Book: the unofficial guide to the worlds most popular disassembler. San Francisco: No Starch Press; 2011.
14. Evtvushkin D, Ponomarev D, Abu-Ghazaleh N. Jump over ASLR: attacking branch predictors to bypass ASLR. In: The 49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-49. IEEE Press; 2016.
15. G SS, Darki A, Faloutsos M, Abu-Ghazaleh N, Sridharan M. Idapro for iot malware analysis? In: 12th USENIX Workshop on Cyber Security Experimentation and Test (CSET 19). USENIX Association, Santa Clara, CA; 2019. <https://www.usenix.org/conference/cset19/presentation/g>. Accessed 8 Oct 2021.
16. Göktas E, Athanasopoulos E, Bos H, Portokalidis G. Out of control: overcoming control-flow integrity. In: 2014 IEEE Symposium on Security and Privacy; 2014. pp. 575–89. <https://doi.org/10.1109/SP.2014.43>.
17. Gras B, Razavi K, Bosman E, Bos H, Giuffrida C. ASLR on the line: practical cache attacks on the MMU. In: 24th Annual Network and Distributed System Security Symposium, NDSS 2017, San Diego, California, USA, February 26–March 1, 2017. The Internet Society; 2017. <https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/aslr-cache-practical-cache-attacks-mmU/>. Accessed 8 Oct 2021.
18. Levinthal D. Performance analysis guide for Intel Core I7 processor and intel xeon 5500 processors. https://software.intel.com/sites/products/collateral/hpc/vtune/performance_analysis_guide.pdf. Accessed: 2018-07-03.
19. Malone C, Zahran M, Karri R. Are hardware performance counters a cost effective way for integrity checking of programs. In: Proceedings of the Sixth ACM Workshop on Scalable Trusted Computing, STC 11, pp. 71–6. Association for Computing Machinery, New York, NY, USA; 2011. <https://doi.org/10.1145/2046582.2046596>.
20. Shacham H, Page M, Pfaff B, Goh EJ, Modadugu N, Boneh D. On the effectiveness of address-space randomization. In: Proceedings of the 11th ACM conference on Computer and communications security; 2004. pp. 298–307. ACM.
21. Shoshitaishvili Y, Wang R, Salls C, Stephens N, Polino M, Dutcher A, Grosen J, Feng S, Hauser C, Kruegel C, Vigna G. Sok: (state of) the art of war: offensive techniques in binary analysis; 2016.
22. Singh B, Evtvushkin D, Elwell J, Riley R, Cervesato I. On the detection of kernel-level rootkits using hardware performance counters. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, ASIA CCS 17, pp. 483–93. ACM, New York, NY, USA; 2017. <https://doi.org/10.1145/3052973.3052999>.
23. Team AD. Ardupilot autopilot suite. URL <http://ardupilot.com/>. Accessed; 2016. pp. 05–20.
24. Team A. ANGR documentation. <https://docs.angr.io/>. Accessed: 2021-05-22.
25. Team L. LIEF documentation. <https://lief.quarkslab.com/>. Accessed: 2021-05-22.
26. Uhsadel L, Georges A, Verbaudhede I. Exploiting hardware performance counters. In: 2008 5th Workshop on Fault Diagnosis and Tolerance in Cryptography; 2008. pp. 59–67.
27. Wang X, Karri R. Numchecker: Detecting kernel control-flow modifying rootkits by using hardware performance counters. In: Proceedings of the 50th Annual Design Automation Conference, DAC 13. Association for Computing Machinery, New York, NY, USA; 2013. <https://doi.org/10.1145/2463209.2488831>.
28. Xia Y, Liu Y, Chen H, Zang B. Cfimon: detecting violation of control flow integrity using performance counters. In: IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012); 2012. pp. 1–12. IEEE.
29. Yuan L, Xing W, Chen H, Zang B. Security breaches as PMU deviation: detecting and identifying security attacks using performance counters. In: Proceedings of the Second Asia-Pacific Workshop on Systems, APSys 11. Association for Computing Machinery, New York, NY, USA; 2011. <https://doi.org/10.1145/2103799.2103807>.
30. Zhou B, Gupta A, Jahanshahi R, Egele M, Joshi A. Hardware performance counters can detect malware: Myth or fact? In: Proceedings of the 2018 on Asia Conference on Computer and Communications Security, ASIACCS 18, pp. 457–68. Association for Computing Machinery, New York, NY, USA; 2018. <https://doi.org/10.1145/3196494.3196515>.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.